Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S004**
Sap ID: 60018250070	**Topic: Data Structure (Queue)**
Branch: Artificial Intelligence & Data Science	

Experiment No. 4

Aim: To implement a Singly Linked List (SLL) that manages a music streaming application's playlist, supporting dynamic operations for adding, deleting, and searching songs at various positions, along with two additional utility operations.

Theory:

Definition:

A singly linked list is a linear dynamic data structure in which each node holds a data field and a single next pointer linking it to the following node. The last node's next holds NULL. Because nodes are allocated individually on the heap, the list grows or shrinks at runtime without contiguous storage or element shifting. Here, each node stores a song name and a pointer to the next song.

Node Structure:-

C Struct:

```
struct Node {
   char songName[100];
   struct Node *next;
};
```

Operations:

AddSongToList

Songs may be inserted at the beginning, at the end, or at any specified position.

1. Add at Beginning

Pseudocode:

```
BEGIN AddAtBeginning(HEAD, SONG_NAME)
  NEW_NODE = ALLOCATE NODE()
  NEW_NODE.songName = SONG_NAME
  NEW_NODE.next = HEAD
  HEAD = NEW_NODE
  PRINT "Song added at the beginning"
END AddAtBeginning
```

2. Add at End

Pseudocode:

```
BEGIN AddAtEnd(HEAD, SONG_NAME)
  NEW_NODE = ALLOCATE NODE()
  NEW_NODE.songName = SONG_NAME
  NEW_NODE.next = NULL
  IF HEAD == NULL THEN
    HEAD = NEW_NODE
  ELSE
    SET TEMP = HEAD
    WHILE TEMP.next != NULL DO
      TEMP = TEMP.next
    END WHILE
    TEMP.next = NEW_NODE
```

```
    END IF
END AddAtEnd
```

3. Add at Position

Pseudocode:

```
BEGIN AddAtPosition(HEAD, SONG_NAME, POS)
    NEW_NODE = ALLOCATE NODE()
    NEW_NODE.songName = SONG_NAME
    IF POS == 1 THEN
        NEW_NODE.next = HEAD
        HEAD = NEW_NODE
    ELSE
        SET TEMP = HEAD
        SET I = 1
        WHILE I < POS - 1 AND TEMP != NULL DO
            TEMP = TEMP.next
            I = I + 1
        END WHILE
        IF TEMP == NULL THEN
            PRINT "Position out of range"
            FREE NEW_NODE
        ELSE
            NEW_NODE.next = TEMP.next
            TEMP.next = NEW_NODE
        END IF
    END IF
END AddAtPosition
```

DeleteSongFromList

Songs may be removed from the beginning, the end, or by matching the song name.

1. Delete at Beginning

Pseudocode:

```
BEGIN DeleteAtBeginning(HEAD)
  IF HEAD == NULL THEN
    PRINT "Playlist is empty"
  ELSE
    TEMP = HEAD
    HEAD = HEAD.next
    PRINT "Deleted: " + TEMP.songName
    FREE TEMP
  END IF
END DeleteAtBeginning
```

2. Delete at End

Pseudocode:

```
BEGIN DeleteAtEnd(HEAD)
  IF HEAD == NULL THEN
    PRINT "Playlist is empty"
  ELSE IF HEAD.next == NULL THEN
    PRINT "Deleted: " + HEAD.songName
    FREE HEAD
    HEAD = NULL
  ELSE
    SET PREV = NULL
```

```
    SET TEMP = HEAD
    WHILE TEMP.next != NULL DO
      PREV = TEMP
      TEMP = TEMP.next
    END WHILE
    PREV.next = NULL
    PRINT "Deleted: " + TEMP.songName
    FREE TEMP
  END IF
END DeleteAtEnd
```

3. Delete by Song Name

Pseudocode:

```
BEGIN DeleteBySongName(HEAD, SONG_NAME)
  IF HEAD == NULL THEN
    PRINT "Playlist is empty"
  ELSE IF HEAD.songName == SONG_NAME THEN
    TEMP = HEAD
    HEAD = HEAD.next
    FREE TEMP
    PRINT "Song deleted"
  ELSE
    SET PREV = HEAD
    SET TEMP = HEAD.next
    WHILE TEMP != NULL DO
      IF TEMP.songName == SONG_NAME THEN
        PREV.next = TEMP.next
```

```
            FREE TEMP
            PRINT "Song deleted"
            RETURN
        END IF
        PREV = TEMP
        TEMP = TEMP.next
    END WHILE
    PRINT "Song not found"
  END IF
END DeleteBySongName
```

SearchSongInTheList

The playlist supports searching by song name (returns position) and by position (returns song name).

1. Search by Song Name

Pseudocode:

```
BEGIN SearchByName(HEAD, SONG_NAME)
  SET TEMP = HEAD
  SET POS = 1
  WHILE TEMP != NULL DO
    IF TEMP.songName == SONG_NAME THEN
      PRINT "Song found at position " + POS
      RETURN POS
    END IF
    TEMP = TEMP.next
    POS = POS + 1
  END WHILE
  PRINT "Song not found"
```

```
    RETURN -1
END SearchByName
```

2. Search by Position

Pseudocode:

```
BEGIN SearchByPosition(HEAD, POS)
    SET TEMP = HEAD
    SET I = 1
    WHILE TEMP != NULL DO
        IF I == POS THEN
            PRINT "Song at position " + POS + ": " + TEMP.songName
            RETURN TEMP
        END IF
        TEMP = TEMP.next
        I = I + 1
    END WHILE
    PRINT "Position out of range"
END SearchByPosition
```

Additional Operations

CountSongs returns the total number of songs; DisplayPlaylist prints every song from head to tail.

1. CountSongs

Pseudocode:

```
BEGIN CountSongs(HEAD)
    SET TEMP = HEAD
    SET COUNT = 0
    WHILE TEMP != NULL DO
```

```
        COUNT = COUNT + 1
        TEMP = TEMP.next
    END WHILE
    PRINT "Total songs: " + COUNT
    RETURN COUNT
END CountSongs
```

2. DisplayPlaylist

Pseudocode:

```
BEGIN DisplayPlaylist(HEAD)
    IF HEAD == NULL THEN
        PRINT "Playlist is empty"
    ELSE
        SET TEMP = HEAD
        WHILE TEMP != NULL DO
            PRINT TEMP.songName + " -> "
            TEMP = TEMP.next
        END WHILE
        PRINT "NULL"
    END IF
END DisplayPlaylist
```

Code:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>

struct Node { char songName[100]; struct Node *next; };
struct Node *head = NULL;

void addAtBeginning() {
   struct Node *newNode = (struct Node*)malloc(sizeof(struct
Node));
   printf("Enter song name: "); gets(newNode->songName);
   newNode->next = head; head = newNode;
   printf("Song added at the beginning.\n");
}

void addAtEnd() {
   struct Node *newNode = (struct Node*)malloc(sizeof(struct
Node)), *temp;
   printf("Enter song name: "); gets(newNode->songName);
   newNode->next = NULL;
   if(head==NULL) head=newNode;
   else { temp=head; while(temp->next!=NULL) temp=temp->next;
temp->next=newNode; }
   printf("Song added at the end.\n");
}

void addAtPosition() {
   struct Node *newNode=(struct Node*)malloc(sizeof(struct
Node)),*temp;
   int pos,i;
   printf("Enter song name: "); gets(newNode->songName);
   printf("Enter position: "); scanf("%d",&pos); getchar();
   if(pos==1){newNode->next=head;head=newNode;}
   else {
      temp=head;
      for(i=1;i<pos-1&&temp!=NULL;i++) temp=temp->next;
      if(temp==NULL){printf("Position out of
range.\n");free(newNode);return;}
      newNode->next=temp->next; temp->next=newNode;
```

```
    }
    printf("Song added at position %d.\n",pos);
}

void deleteAtBeginning() {
    struct Node *temp;
    if(head==NULL){printf("Playlist is empty.\n");return;}
    temp=head; head=head->next;
    printf("Deleted: %s\n",temp->songName); free(temp);
}

void deleteAtEnd() {
    struct Node *temp,*prev;
    if(head==NULL){printf("Playlist is empty.\n");return;}
    if(head->next==NULL){printf("Deleted: %s\n",head-
>songName);free(head);head=NULL;return;}
    temp=head; prev=NULL;
    while(temp->next!=NULL){prev=temp;temp=temp->next;}
    prev->next=NULL; printf("Deleted: %s\n",temp->songName);
free(temp);
}

void deleteBySong() {
    struct Node *temp,*prev; char name[100];
    printf("Enter song name to delete: "); gets(name);
    if(head==NULL){printf("Playlist is empty.\n");return;}
    if(strcmp(head->songName,name)==0){temp=head;head=head-
>next;free(temp);printf("Song deleted.\n");return;}
    prev=head; temp=head->next;
    while(temp!=NULL){
        if(strcmp(temp->songName,name)==0){prev->next=temp-
>next;free(temp);printf("Song deleted.\n");return;}
        prev=temp; temp=temp->next;
    }
    printf("Song not found.\n");
}

void searchByName() {
    struct Node *temp; char name[100]; int pos=1;
    printf("Enter song name: "); gets(name); temp=head;
    while(temp!=NULL){
```

```
        if(strcmp(temp->songName,name)==0){printf("Found at
position %d.\n",pos);return;}
        temp=temp->next; pos++;
    }
    printf("Song not found.\n");
}

void searchByPosition() {
    struct Node *temp; int pos,i;
    printf("Enter position: "); scanf("%d",&pos); getchar();
temp=head;
    for(i=1;i<pos&&temp!=NULL;i++) temp=temp->next;
    if(temp==NULL) printf("Position out of range.\n");
    else printf("Song: %s\n",temp->songName);
}

int countSongs() {
    struct Node *temp=head; int count=0;
    while(temp!=NULL){count++;temp=temp->next;} return count;
}

void displayPlaylist() {
    struct Node *temp;
    if(head==NULL){printf("Playlist is empty.\n");return;}
    temp=head; printf("Playlist: ");
    while(temp!=NULL){printf("%s -> ",temp-
>songName);temp=temp->next;}
    printf("NULL\n");
}

void main() {
    int choice; clrscr();
    do {
        printf("\n--- Music Playlist Manager ---\n");
        printf("1.Add Begin 2.Add End 3.Add Pos\n4.Del Begin 5.Del
End 6.Del Name\n");
        printf("7.Search Name 8.Search Pos 9.Count 10.Display
0.Exit\n");
        printf("Choice: "); scanf("%d",&choice); getchar();
        switch(choice){
            case 1:addAtBeginning();break; case 2:addAtEnd();break;
```

```
            case 3:addAtPosition();break; case
        4:deleteAtBeginning();break;
            case 5:deleteAtEnd();break;   case 6:deleteBySong();break;
            case 7:searchByName();break;  case
        8:searchByPosition();break;
            case 9:printf("Total: %d\n",countSongs());break;
            case 10:displayPlaylist();break; case 0:break;
            default:printf("Invalid.\n");
          }
       } while(choice!=0);
       getch();
    }
```

Output:

```
--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name: Song added at the end.


--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
```

```
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name: Song added at the end.

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name: Song added at the end.

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
```

```
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Playlist: Bohemian Rhapsody -> Hotel California -> Stairway to Hea
ven -> NULL


--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Total: 3


--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
```

```
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name: Found at position 2.

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter position: Song: Hotel California

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
```

6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name: Song added at the beginning.

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name: Enter position: Song added at position 3.

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name

7.Search Name

8.Search Pos

9.Count

10.Display

0.Exit

Choice: Playlist: Shape of You -> Bohemian Rhapsody -> Imagine -> Hotel California -> Stairway to Heaven -> NULL

--- Music Playlist Manager ---

1.Add Begin

2.Add End

3.Add Pos

4.Del Begin

5.Del End

6.Del Name

7.Search Name

8.Search Pos

9.Count

10.Display

0.Exit

Choice: Enter song name: Found at position 3.

--- Music Playlist Manager ---

1.Add Begin

2.Add End

3.Add Pos

4.Del Begin

5.Del End

6.Del Name

7.Search Name

8.Search Pos

9.Count

10.Display

0.Exit

Choice: Deleted: Shape of You

--- Music Playlist Manager ---

1.Add Begin

2.Add End

3.Add Pos

4.Del Begin

5.Del End

6.Del Name

7.Search Name

8.Search Pos

9.Count

10.Display

0.Exit

Choice: Playlist: Bohemian Rhapsody -> Imagine -> Hotel California -> Stair way to Heaven -> NULL

--- Music Playlist Manager ---

1.Add Begin

2.Add End

3.Add Pos

4.Del Begin

5.Del End

6.Del Name

```
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Deleted: Stairway to Heaven

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Playlist: Bohemian Rhapsody -> Imagine -> Hotel California -> NUL
L

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
```

```
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Enter song name to delete: Song deleted.

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Playlist: Imagine -> Hotel California -> NULL

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
```

8.Search Pos
9.Count
10.Display
0.Exit
Choice: Deleted: Imagine

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos
9.Count
10.Display
0.Exit
Choice: Deleted: Hotel California

--- Music Playlist Manager ---
1.Add Begin
2.Add End
3.Add Pos
4.Del Begin
5.Del End
6.Del Name
7.Search Name
8.Search Pos

```
9.Count
10.Display
0.Exit
Choice: 0
```

Applications:

Singly linked lists are used in music streaming playlists, web browser back-navigation history, OS process scheduling queues, compiler symbol tables, and text editor undo chains.

Conclusion:

This experiment implemented a singly linked list to manage a dynamic music playlist. All ten operations were coded in C, reinforcing the advantages of linked lists over arrays: O(1) head insertion without element shifting, and dynamic heap allocation without a fixed-capacity constraint.